Project 2:-Maven Project with Guava and Commons-IO Dependencies

1:Create a Maven Project Using Terminal by following command:

mvn archetype:generate -DgroupId=com.example -DartifactId=MyMavenGuavaApp -DarchetypeArtifactId=maven-archetype-quickstart -DinteractiveMode=false

```
brijesh@Ubuntu:~/devops$ mvn archetype:generate -DgroupId=com.example -DartifactId=MyMavenGuavaApp -DarchetypeArtifactId=maven
[INFO] Scanning for projects...
[INFO]
[INFO] ------------------< org.apache.maven:standalone-pom >-------------------
[INFO] Building Maven Stub Project (No POM) 1
[INFO] --------------------------------[ pom ]---------------------------------
[INFO]
[INFO] >>> maven-archetype-plugin:3.4.1:generate (default-cli) > generate-sources @ standalone-pom >>>
[INFO]
[INFO] <<< maven-archetype-plugin:3.4.1:generate (default-cli) < generate-sources @ standalone-pom <<<
[INFO]
[INFO]
[INFO] --- maven-archetype-plugin:3.4.1:generate (default-cli) @ standalone-pom ---
[INFO] Generating project in Batch mode
[INFO] No archetype defined. Using maven-archetype-quickstart (org.apache.maven.archetypes:maven-archetype-quickstart:1.0)
[INFO] ----------------------------------------------------------------------------
[INFO] Using following parameters for creating project from Old (1.x) Archetype: maven-archetype-quickstart:1.0
[INFO] ----------------------------------------------------------------------------
[INFO] Parameter: basedir, Value: /home/brijesh/devops
[INFO] Parameter: package, Value: com.example
[INFO] Parameter: groupId, Value: com.example
[INFO] Parameter: artifactId, Value: MyMavenGuavaApp
[INFO] Parameter: packageName, Value: com.example
[INFO] Parameter: version, Value: 1.0-SNAPSHOT
[INFO] project created from Old (1.x) Archetype in dir: /home/brijesh/devops/MyMavenGuavaApp
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  3.443 s
[INFO] Finished at: 2026-03-21T12:57:36+05:30
[INFO] ------------------------------------------------------------------------
brijesh@Ubuntu:~/devops$
```

Now Navigate to the Project Directory using cd MyMavenGuavaApp

Then, use tree command to check the directory structure created by our command

```
brijesh@Ubuntu:~/devops$ cd MyMavenGuavaApp
brijesh@Ubuntu:~/devops/MyMavenGuavaApp$ tree
.
├── pom.xml
└── src
    ├── main
    │   └── java
    │       └── com
    │           └── example
    │               └── App.java
    └── test
        └── java
            └── com
                └── example
                    └── AppTest.java

9 directories, 3 files
brijesh@Ubuntu:~/devops/MyMavenGuavaApp$
```

2: Edit the pom.xml File to Add Dependencies using gedit pom.xml

Open pom.xml ~/DevOps/MyMavenGuavaApp Save

```xml
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/maven-v4_0_0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <groupId>com.example</groupId>
  <artifactId>MyMavenGuavaApp</artifactId>
  <packaging>jar</packaging>
  <version>1.0-SNAPSHOT</version>
  <name>MyMavenGuavaApp</name>
  <url>http://maven.apache.org</url>
  <dependencies>
    <dependency>
      <groupId>junit</groupId>
      <artifactId>junit</artifactId>
      <version>3.8.1</version>
      <scope>test</scope>
    </dependency>

    <!-- https://mvnrepository.com/artifact/com.google.guava/guava -->
<dependency>
    <groupId>com.google.guava</groupId>
    <artifactId>guava</artifactId>
    <version>33.4.0-jre</version>
</dependency>

<!-- https://mvnrepository.com/artifact/commons-io/commons-io -->
<dependency>
    <groupId>commons-io</groupId>
    <artifactId>commons-io</artifactId>
    <version>2.18.0</version>
</dependency>

  </dependencies>

  <!-- Build: Configuring plugins and build settings -->
<build>
<plugins>
<!-- Example: Maven Compiler Plugin to compile Java code -->
<plugin>
<groupId>org.apache.maven.plugins</groupId>
<artifactId>maven-compiler-plugin</artifactId>
<version>3.8.1</version>
<configuration>
<source>1.7</source>
<target>1.7</target>
```

XML Tab Width: 8 Ln 71, Col 11 INS

```xml
<source>1.7</source>
<target>1.7</target>
</configuration>
</plugin>

<!-- Example: Maven Surefire Plugin to run tests -->
<plugin>
<groupId>org.apache.maven.plugins</groupId>
<artifactId>maven-surefire-plugin</artifactId>
<version>2.22.2</version>
</plugin>


<plugin>
            <groupId>org.apache.maven.plugins</groupId>
            <artifactId>maven-jar-plugin</artifactId>
            <version>3.1.0</version>
            <configuration>
                <archive>
                    <manifestEntries>
                        <Main-Class>com.example.App</Main-Class>
                    </manifestEntries>
                </archive>
            </configuration>
        </plugin>

</plugins>
</build>
</project>
```

XML Tab Width: 8 Ln 71, Col 11 INS

3:Modify App.java

Got to App.java using cd src/main/java/com/example/ and gedit App.java to type the file with following code

```java
import java.io.File;

// Import IOException to handle input-output errors
import java.io.IOException;

// Import ImmutableList from Google Guava library
// ImmutableList creates a list that cannot be modified after creation
import com.google.common.collect.ImmutableList;

// Define the main class
public class App
{
    // Main method - program execution starts from here
    public static void main( String[] args )
    {
        // Create an immutable list of strings (fruits)
        ImmutableList<String> fruits = ImmutableList.of("Apple", "Banana", "Cherry");

        // Print the list of fruits to the console
        System.out.println(fruits);

        // Create a File object representing the source file
        File sourceFile = new File("source.txt");

        // Create a File object representing the destination file
        File destFile = new File("destination.txt");

        try {
            // Copy the contents of source.txt to destination.txt
            // Using Apache Commons IO FileUtils class
            FileUtils.copyFile(sourceFile, destFile);

            // Print success message if file is copied
            System.out.println("File copied successfully!");
        }
        catch (IOException e) {
            // If any error occurs during file copying,
            // this block will handle the exception

            // Print error message
            System.err.println("Error occurred while copying file: " + e.getMessage());
        }
    }
}
```

Build the Project using the command mvn compile command

```
brijesh@Ubuntu:~/Devops$ cd MyMavenGuavaApp
brijesh@Ubuntu:~/Devops/MyMavenGuavaApp$ mvn compile
[INFO] Scanning for projects...
[INFO]
[INFO] --------------------< com.example:MyMavenGuavaApp >--------------------
[INFO] Building MyMavenGuavaApp 1.0-SNAPSHOT
[INFO] --------------------------------[ jar ]---------------------------------
[INFO]
[INFO] --- maven-resources-plugin:2.6:resources (default-resources) @ MyMavenGuavaApp ---
[WARNING] Using platform encoding (UTF-8 actually) to copy filtered resources, i.e. build is platform dependent!
[INFO] skip non existing resourceDirectory /home/brijesh/Devops/MyMavenGuavaApp/src/main/resources
[INFO]
[INFO] --- maven-compiler-plugin:3.8.1:compile (default-compile) @ MyMavenGuavaApp ---
[INFO] Changes detected - recompiling the module!
[WARNING] File encoding has not been set, using platform encoding UTF-8, i.e. build is platform dependent!
[INFO] Compiling 1 source file to /home/brijesh/Devops/MyMavenGuavaApp/target/classes
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  3.172 s
[INFO] Finished at: 2026-03-21T13:13:01+05:30
[INFO] ------------------------------------------------------------------------
brijesh@Ubuntu:~/Devops/MyMavenGuavaApp$
```

Now, Create a Sample File to Copy

Run the Application:

If java-jar command give an error which is most probable, use the following command

mvn exec:java -Dexec.mainClass=”com.example.App”

```
brijesh@Ubuntu:~/Devops/MyMavenGuavaApp$ mvn exec:java -Dexec.mainClass="com.example.App"
[INFO] Scanning for projects...
[INFO]
[INFO] -------------------< com.example:MyMavenGuavaApp >--------------------
[INFO] Building MyMavenGuavaApp 1.0-SNAPSHOT
[INFO] --------------------------------[ jar ]---------------------------------
[INFO]
[INFO] --- exec-maven-plugin:3.6.3:java (default-cli) @ MyMavenGuavaApp ---
[Apple, Banana, Cherry]
File copied successfully!
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  1.157 s
[INFO] Finished at: 2026-03-21T13:26:33+05:30
[INFO] ------------------------------------------------------------------------
brijesh@Ubuntu:~/Devops/MyMavenGuavaApp$ gedit destination.txt
brijesh@Ubuntu:~/Devops/MyMavenGuavaApp$
```

The output shows file copied successfully.

We can see newly created destination.txt using gedit destination.txt if you are still in Project’s directory

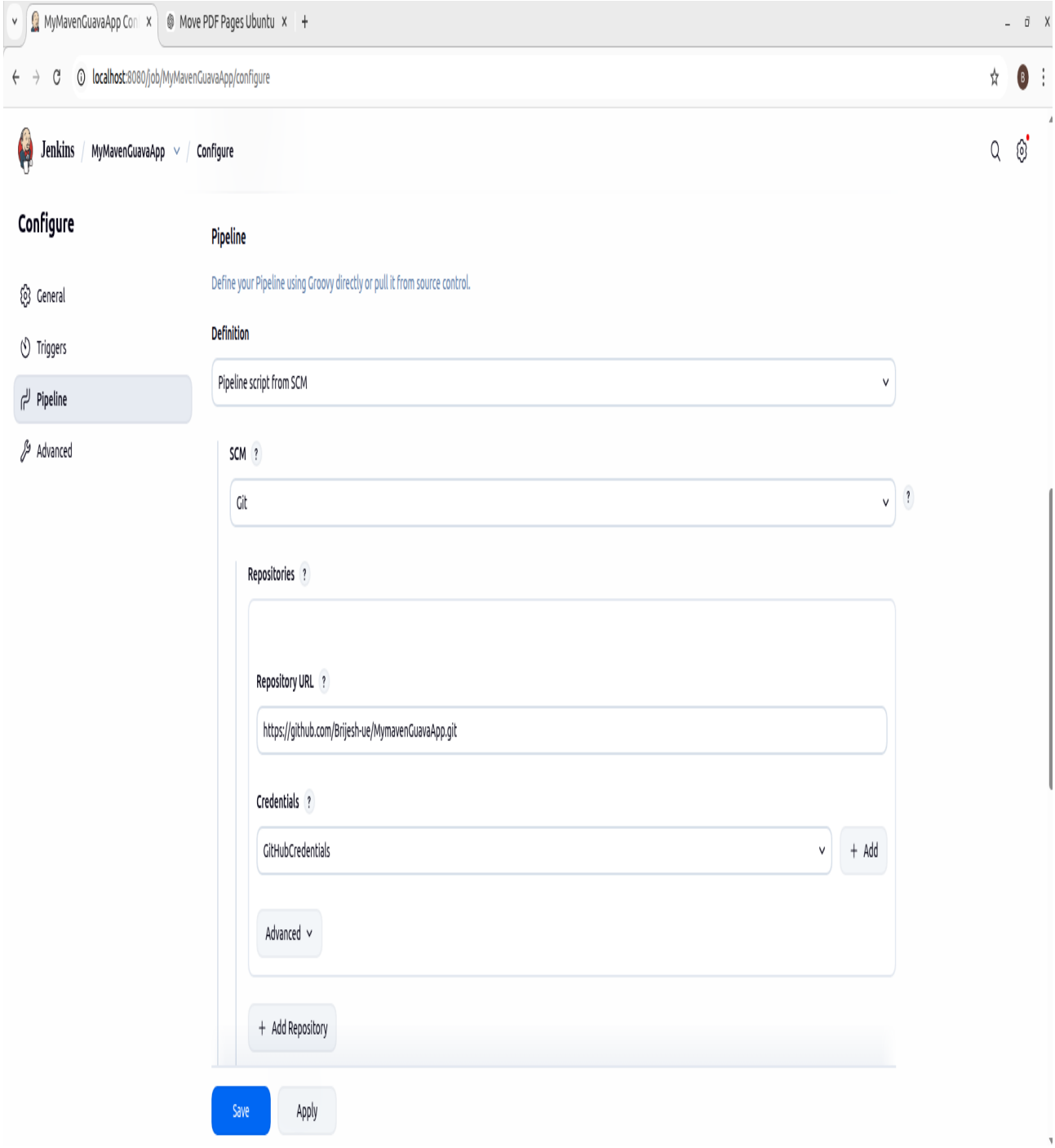
MyMavenGuavaApp Pipeline:

Step 1: Create a New Pipeline Job

Go to Dashboard → + New Item.
Enter name, select Pipeline, and click OK.

Step 2:Configure Pipeline from SCM

Scroll down and select Pipeline script from SCM.
SCM: Git Enter repository URL and select GitHub credentials.

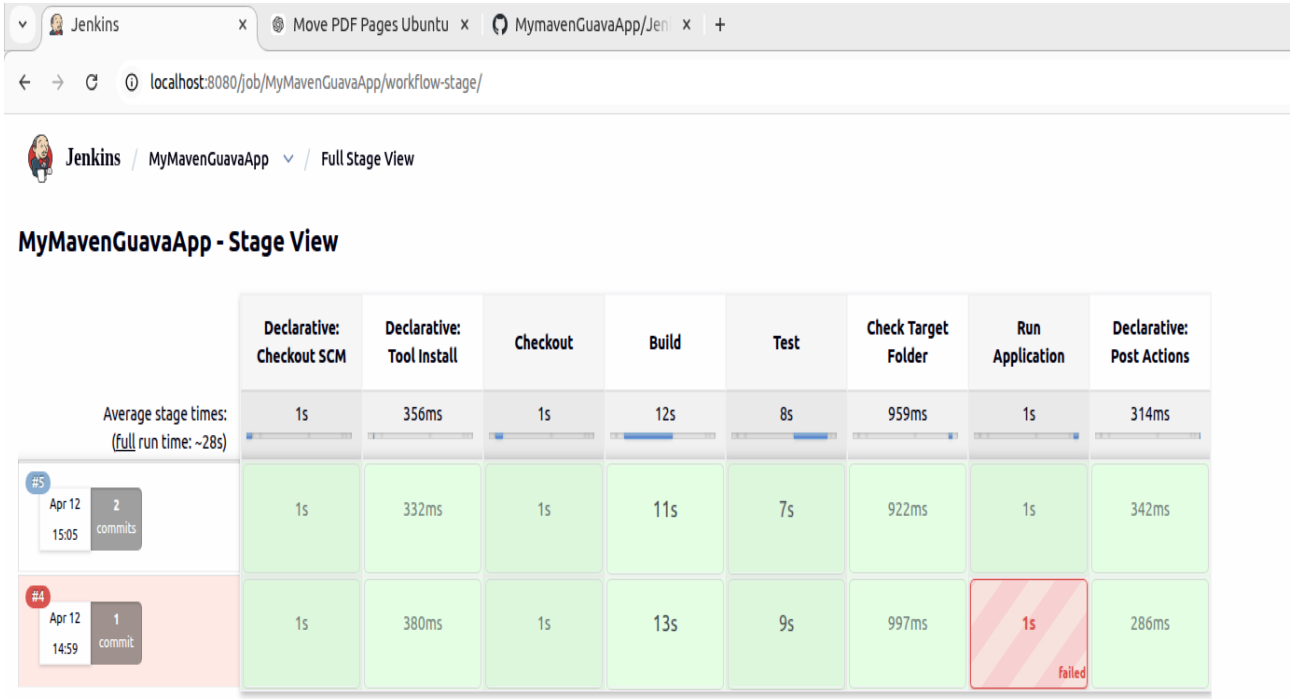


Step 3: Create Jenkinsfile

```
pipeline {
    agent any  // Use any available agent

    tools {
        maven 'Maven'  // Ensure this matches the name configured in Jenkins
    }
    stages {
        stage('Checkout') {
            steps {
                git branch: 'master', url:
'https://github.com/Brijesh-ue/MymavenGuavaApp.git'
            }
        }

        stage('Build') {
            steps {
                sh 'mvn clean package'  // Run Maven build
            }
        }

        stage('Test') {
            steps {
                sh 'mvn test'  // Run unit tests
            }
        }

stage('Check Target Folder') {
    steps {
        sh 'ls -l target'
    }
}



        stage('Run Application') {
            steps {
                // Start the JAR application
```

```
            sh 'java -jar target/MyMavenGuavaApp-1.0-SNAPSHOT.jar'
        }
    }


}

post {
    success {
        echo 'Build and deployment successful!'
    }
    failure {
        echo 'Build failed!'
    }
}
}
```

Step 4: Build the Pipeline

Go to Jenkins and click Build Now.

Monitor Build Stages Use Stage View and Full Stage View to monitor execution.